

# AI Learning & Study Assistant

*Agentic AI Project Report – Naan Mudhalvan Program*

---

|                |   |                             |              |   |       |
|----------------|---|-----------------------------|--------------|---|-------|
| Student Name   | : | _____                       | Register No. | : | _____ |
| Department     | : | _____                       | College      | : | _____ |
| Course / Batch | : | Agentic AI - Naan Mudhalvan | Date         | : | _____ |

## 1. Project Overview

### Project Title

AI Learning & Study Assistant

### Problem Statement

Students today have access to an overwhelming volume of course material – lecture notes, textbook chapters, slides, and reference PDFs – but very little structured guidance on how to convert that material into an effective study routine. Manually building a day-by-day revision schedule, re-reading long documents to find answers to specific doubts, and preparing self-assessment questions before an exam are all time-consuming tasks that most students do inconsistently, if at all. This often results in last-minute, unstructured cramming instead of steady, topic-wise preparation. There is a genuine need for an intelligent, always-available assistant that can read a student's own course material, answer questions strictly from that material, and actively guide the student's learning journey through personalized plans and practice tests, rather than offering generic, one-size-fits-all content pulled from the open internet.

### Brief Description

The AI Learning & Study Assistant is an agentic AI application designed to help students learn more effectively and independently. It allows a student to upload their own notes or textbook excerpts, after which the assistant can answer questions grounded strictly in that material, generate a personalized multi-day study or revision plan based on the topics involved, and create multiple-choice quizzes with explanations to test understanding. The system combines three agentic building blocks – Retrieval-Augmented Generation (RAG) to fetch relevant course content, a persistent Memory layer to remember what a student has studied and where they are weak, and a set of callable Tools that perform concrete actions such as plan or quiz generation. All reasoning and generation is performed by a large language model running locally through Ollama, which means the assistant can operate without depending on a paid cloud API and keeps the student's academic data on their own machine.

## 2. Objectives & Proposed Solution

### Project Objectives

- Help students create structured, personalized study and revision plans that fit the amount of time they actually have available before an exam or deadline.
- Provide accurate, trustworthy answers to student questions that are grounded strictly in their own uploaded course material, rather than generic or possibly incorrect web content.

- Generate topic-wise quizzes and practice questions, complete with explanations, so students can actively test and reinforce their understanding instead of passively re-reading notes.
- Track each student's topic history, quiz performance and weak areas across multiple sessions so that later interactions with the assistant are increasingly personalized.
- Demonstrate a practical, end-to-end agentic AI system that runs entirely on a locally hosted open-source LLM through Ollama, without dependence on any paid external AI service.

## How the Agentic AI Solution Works

The system is built around a large language model served locally through **Ollama**, orchestrated by three cooperating agentic components rather than a single monolithic prompt. A **Retrieval-Augmented Generation (RAG)** pipeline is responsible for turning raw course material into searchable knowledge: uploaded text is split into small chunks, each chunk is converted into a vector embedding using an Ollama embedding model, and all embeddings are stored in a **ChromaDB** vector database. When a student asks a question, the same embedding process is applied to the query, the most semantically similar chunks are retrieved, and only that relevant context is passed to the LLM, which keeps answers accurate and grounded rather than hallucinated. A **Memory** module, implemented as a lightweight JSON store, keeps a running record of every topic the student has touched, every quiz they have attempted with their score, and any topic where their score fell below an acceptable threshold, which is then flagged as a weak area. A set of **Tools** wraps specific, reusable actions – generating a day-wise study plan, generating a structured multiple-choice quiz, and answering a grounded question – as separate callable functions rather than a single free-form response. Finally, an **Agent** layer reads the student's incoming message, performs simple intent detection to decide whether the student wants a quiz, a study plan, or an answer to a question, retrieves the necessary context and memory, calls the matching tool, and returns a clean, structured response back to the student while updating memory in the background.

## Key Features

- **Course material ingestion:** students can upload notes, textbook excerpts or slide content, which is automatically chunked and indexed for retrieval.
- **Personalized study plans:** day-wise revision schedules are auto-generated based on the topics a student has already covered and the study time they have available.
- **On-demand quiz generation:** multiple-choice questions are generated on any covered topic, each with four options, the correct answer, and a short explanation.
- **Persistent progress tracking:** topics covered, quiz scores and identified weak areas are stored across sessions so guidance improves the more the assistant is used.
- **Grounded conversational Q&A:** answers are generated strictly from the retrieved course material, and the assistant is instructed to say so honestly when an answer is not present in the material, which greatly reduces hallucination.

## 3. Implementation & Results

### Technologies / Tools Used

- **Ollama** – runs the local large language model (e.g. Llama 3) that powers both chat responses and the text embeddings used for retrieval, with no external API dependency.

- **ChromaDB** – an open-source vector database used to store chunk embeddings and perform fast similarity search for the RAG pipeline.
- **Python** – the core application language, used to implement the agent logic, tool functions, and orchestration between RAG, memory and the LLM.
- **JSON file store** – a lightweight, dependency-free persistent memory layer that records topics covered, quiz scores and weak areas between sessions.
- **Regex and JSON parsing** – used to reliably extract structured quiz data (a list of questions, options, answers and explanations) from the LLM's free-form text response.

## Working Process

- The student uploads their course material, such as lecture notes or extracted PDF text, into the assistant.
- The material is automatically split into small, overlapping chunks and embedded into the ChromaDB vector store so that it can later be searched semantically.
- The student interacts with the assistant through a simple chat interface, typing a normal question, or asking directly for a quiz or a study plan.
- The agent layer detects the intent of the message, retrieves the most relevant chunks of course material through RAG, checks the student's memory of past sessions, and invokes the matching tool – the Q&A tool, the quiz generator, or the study-plan generator.
- The Ollama-hosted LLM generates the actual response text, which is then parsed – for example, into structured JSON in the case of a quiz – and displayed back to the student in a clean, readable format.
- Finally, the student's newly covered topics, quiz results and any weak areas identified during the session are written back into the persistent memory store, so that the next session can build on what has already happened rather than starting from zero.

## Screenshot / Output

```
python ai_learning_assistant.py

$ python ai_learning_assistant.py

[RAG] Indexed 1 chunks from biology_ch4.txt

--- Q&A ---
Photosynthesis is the process by which green plants convert light energy into chemical energy stored as glucose. It occurs in the chloroplasts, using chlorophyll to absorb sunlight, and involves two stages: the light-dependent reactions (thylakoid membrane) and the Calvin cycle (stroma).

--- Quiz ---
[
  {
    "question": "Where does photosynthesis primarily occur?",
    "options": ["A. Mitochondria", "B. Chloroplasts", "C. Nucleus", "D. Ribosomes"],
    "answer": "B",
    "explanation": "Chloroplasts contain chlorophyll, which absorbs sunlight."
  },
  {
    "question": "What gas is released during light-dependent reactions?",
    "options": ["A. CO2", "B. Nitrogen", "C. Oxygen", "D. Hydrogen"],
    "answer": "C",
    "explanation": "Oxygen is released as a byproduct of splitting water."
  }
]

--- Study Plan ---
Day 1: Overview of photosynthesis - light vs. dark reactions (2 hrs)
Day 2: Deep dive - light-dependent reactions & thylakoid membrane (2 hrs)
Day 3: Calvin cycle & stroma processes (2 hrs)
Day 4: Practice diagrams + quiz review of weak areas (2 hrs)
Day 5: Full topic revision + mock quiz (2 hrs)

--- Memory Summary ---
{
  "topics_covered": ["photosynthesis", "What is photosynthesis and why?"],
  "quiz_scores": {},
  "weak_areas": []
}
```

## Results Achieved

- The assistant successfully answered course-related questions with source-grounded accuracy, using RAG to restrict its responses to the student's own uploaded material.
- It generated coherent, personalized multi-day study plans that reflected the topics already covered by the student and the amount of daily study time they specified.
- It produced relevant, well-formed multiple-choice quiz questions, each with four options, a marked correct answer, and a short explanation, enabling immediate self-assessment.
- It demonstrated genuine memory retention across sessions, correctly recalling previously covered topics and flagging any topic with a low quiz score as a weak area for revision.
- The complete pipeline – ingestion, retrieval, reasoning, tool-calling and memory update – ran end-to-end on a fully local setup, confirming that a capable agentic study assistant can be built without any paid cloud AI service.

## 4. Conclusion & Future Scope

### Conclusion

The AI Learning & Study Assistant demonstrates how agentic AI – combining retrieval-augmented generation, persistent memory, and structured tool-calling on a locally hosted LLM served through Ollama – can meaningfully support students in their day-to-day learning. Rather than relying on generic, one-size-fits-all content pulled from the open internet, the assistant grounds every answer in the student's own course material and continuously adapts its guidance based on their actual progress and weak areas. The project shows, in a practical and reproducible way, that the combination of RAG, memory and tools around an open-source LLM is sufficient to build a genuinely useful, privacy-respecting educational agent, making studying more structured, more personalized, and more interactive for the end user.

### Challenges Faced

- Ensuring accurate and complete retrieval from diverse and sometimes poorly formatted course materials, such as notes with inconsistent headings or scanned content.
- Reliably parsing structured JSON quiz output from what is fundamentally a free-form text response from the LLM, which occasionally required extra prompt tuning and regex handling.
- Balancing the difficulty of generated quiz questions so that they genuinely match the student's actual level of understanding rather than being too easy or too obscure.
- Maintaining useful context and memory across long study sessions and multiple topics without the local model's performance or response quality degrading.

### Future Enhancements

- Adding voice-based interaction so students can study hands-free while commuting or reviewing flashcards.
- Integrating directly with LMS platforms such as Google Classroom or Moodle so course material can be auto-synced instead of manually uploaded.
- Introducing adaptive-difficulty quizzes that adjust question difficulty in real time based on ongoing performance analytics stored in memory.
- Building a full web-based chat interface, for example with Streamlit, to make the assistant more accessible and visually engaging for everyday student use.

- Adding multi-language support so the assistant can serve a wider and more diverse group of students in their preferred language.

### Key Learning Outcomes

Building this project provided hands-on experience across the full agentic AI stack rather than just prompting a single chatbot. It involved setting up and running a large language model entirely locally through Ollama, designing a retrieval-augmented generation pipeline with chunking and vector search using ChromaDB, engineering prompts that reliably return structured JSON output for downstream parsing, and designing a simple but effective persistent memory layer to support personalization across sessions. It also reinforced practical software engineering skills such as modular code design, where the RAG layer, the memory layer, and the tool functions are kept independent and easily testable, as well as an appreciation for why grounding an LLM's answers in real source material is essential for building trustworthy, education-focused AI applications.

### References

- Ollama Documentation – <https://ollama.com>
- ChromaDB Documentation – <https://docs.trychroma.com>
- Lewis, P. et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.
- Naan Mudhalvan – Agentic AI Course Material, TN Government Skill Development Scheme.
- Python, ChromaDB and Ollama official GitHub repositories, accessed during project development for API usage and configuration reference.

---

*Declaration: This project report is submitted as part of the Naan Mudhalvan – Agentic AI programme and represents original work carried out by the undersigned student.*

Student Signature: \_\_\_\_\_

Faculty / Mentor Signature:

\_\_\_\_\_